Detection of Faults in Microservices using Petri Nets

Sulochan Naik
sulochan.naik@iiitb.org
International Institute of Information Technology
Bengaluru, Karnataka, India

Meenakshi D'Souza
meenakshi@iiitb.ac.in
International Institute of Information Technology
Bengaluru, Karnataka, India

ABSTRACT

Despite higher adoption of Microservices worldwide, there is a lack of work on possible fault detection during early phases such as at requirements and design phase. We propose a novel way to detect faults using model checking CTL specifications and a SMT solver. Requirements of a system involving microservices are mapped to Netflix's Conductor specification which is later converted into Petri nets for further analysis. Traversal analysis on the converted Petri nets not only allows us to detect a faulty specification but to correct it and adopt a better design. We tried our methodology on the TrainTicket benchmark system and we are able to detect seven faults from the benchmark application using our approach.

KEYWORDS

Microservices, Fault detection, Petri Nets, Conductor, Benchmark

ACM Reference Format:
Sulochan Naik and Meenakshi D'Souza. 2023. Detection of Faults in Microservices using Petri Nets. In *16th Innovations in Software Engineering Conference (ISEC 2023), February 23–25, 2023, Allahabad, India.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3578527.3578541

1 INTRODUCTION

Microservices adoption has become a new industry standard owing to its inherent properties and advantages. According to a 2021 statistics [15], microservices had an adoption rate of 37% worldwide. Increased adoption immediately implies a need for better maintenance and error handling. Detection of faults in microservices plays a major role in an effective functioning of any service oriented industry. If faults are detected after deployment, resources go wasted in unnecessary postmortem analysis. Designers strive to detect faults early, at requirements or design phase. In this paper, we propose a novel way to detect possible faults using traversal analysis of Petri nets derived from microservice requirements.

We use Netflix's Conductor orchestration [12, 13] as a base to achieve our solution. Requirements specification of any microservice application is initially mapped to Conductor workflow specification. The conductor workflow is later converted to Petri nets. Traversal analysis on this converted Petri nets is performed to derive various properties like deadlock, liveness and bounded response time as CTL formulas for simple systems and for complex systems using Z3 solver. Using these derived properties, faults for microservices are identified.

We have identified faults using our methodology and compared them with actual faults detected by different means on a popular faults benchmark application, TrainTicket [11]. We write our own Conductor specification based on the architecture of Trainticket system. Task definitions and workflow are meticulously captured as a part of the specification, based on architecture. We then use Conductor2PN [1] to convert a Trainticket specification into its equivalent Petri net. We further use traversal analysis on these Petri nets to identify faults and then compare with the actual faults detected and reported in the benchmark.

Rest of the paper is divided into the following sections. Section 2 presents a brief background on microservices, Conductor specification, Petri Nets and computation tree logic (CTL). Section 3 contains related work in the area of microservices fault detection. Section 4 explains our methodology which is followed by results and analysis in Section 5. We conclude in Section 6.

2 BACKGROUND

2.1 Microservices

The microservice architectural style is an approach to develop a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API [8]. These services are built around business capabilities and are independently deployable by a fully automated deployment machinery [8]. There is a bare minimum of centralised management of these services, which may be written in different programming languages and use different data storage technologies [8]. Componentization via services, decentralised governance, decentralised data management and infrastructure automation are various characteristics of microservices. Microservices solved various problems faced by monolithic architecture in terms of both implementation and maintainability.

2.2 Netfilx Conductor specification for microservices

Conductor is a workflow orchestration engine that runs in the cloud [12]. It is a JSON-based language designed by Netflix and is used to specify microservices process flows. Tasks and workflows form essential components of a conductor. Tasks are the building blocks of workflow in a conductor. A task can be an operator, system task or custom code written in any programming language while on the other hand, a workflow is the conductor primitive that encompasses the definition and flow of business logic that a microservice implements [12]. TrainTicket is a popular benchmark microservice application [11], released with a documentation of faults that were detected in the application. Our experiments are based on

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ISEC 2023, February 23–25, 2023, Allahabad, India
© 2023 Association for Computing Machinery.
ACM ISBN 979-8-4007-0064-4/23/02…$15.00
https://doi.org/10.1145/3578527.3578541

```
[
  {
    "name": "cancel_service",
    "retryCount": 3,
    "retryLogic": "FIXED",
    "retryDelaySeconds": 10,
    "timeoutSeconds": 300,
    "timeoutPolicy": "TIME_OUT_WF",
    "responseTimeoutSeconds": 180
  },
  {
    "name": "payment_service",
    "retryCount": 3,
    "retryLogic": "FIXED",
    "retryDelaySeconds": 10,
    "timeoutSeconds": 300,
    "timeoutPolicy": "TIME_OUT_WF",
    "responseTimeoutSeconds": 180
  },
  {
    "name": "order_service",
    "retryCount": 3,
    "retryLogic": "FIXED",
    "retryDelaySeconds": 10,
    "timeoutSeconds": 300,
    "timeoutPolicy": "TIME_OUT_WF",
    "responseTimeoutSeconds": 180
  }
]
```

Figure 1: Tasks Definition for part of TrainTicket service using Conductor

this benchmark example. Figures 1 and 2 provide some of the tasks and workflow definitions for cancel service, payment service and order service respectively which are used further in Section 4 where properties are identified. These specifications will be used for fault detection as explained in Section 4. Various keywords in the task definition like `retry`, `timeout` and response time are self explanatory considering the nature of microservices which operate over different distributed networks.

2.3 Petri Nets

A Petri net is a mathematical modeling language for describing distributed systems [14]. It contains places and transitions, tokens are contained within places. A transition is enabled if a place contains at least one token. There are various variants of Petri nets. Adding a time attribute to simple Petri net would convert it into a timed Petri net. Example of a Petri net is shown in Figure 3 where components marked with P are places, with T are transitions and the black dots within places are tokens. We can see that transition T1 is enabled in Figure 3 since place P1 contains a token. Similarly other transitions are also enabled based on existence of tokens.

2.4 Conductor2PN

Conductor2PN is a Java tool that translates Conductor specifications into a timed basic Petri net model [5]. The translated Petri net can be model checked using any of the popular model checking tools applicable for Petri nets [3, 6].

2.5 Computation Tree Logic (CTL)

CTL is a branching-time propositional temporal logic where formulas are defined using the operators X (next point in the future), U (until), F (some point in the future) and G (everywhere in the future) and quantifiers E (there exists) and A (for all). A model is a tree of computation paths [2] of a system over which the formulas are evaluated. For a detailed explanation of the syntax and semantics of CTL, the reader is referred to [2]. Formulas in CTL can be used to specify properties that hold over one path, over all paths, starting at some point in the future or at all points in the future.

3 RELATED WORK

In [7], microservices faults are detected by monitoring and analyzing real-time performance data. In [9], faults are detected by comparing execution traces. Faults are detected in [17] by analysing system trace logs. [16] employs principal component analysis to extract suspicious microservices with large fluctuations in response time. Most of the research is oriented towards improving quality of service and fault detection at run time using traces. To the extent we are aware of, there is no work on microservices analysis at requirements level and our work leverages this by finding faults at early stage and preventing them from going down to latter stages of development.

4 METHODOLOGY

Figure 4 gives a description of our proposed methodology. Initially, we consider as input, any microservice specification for which faults are to be identified. We convert the given specification into an equivalent one which matches the Conductor syntax and semantics. This is currently a manual process converted through the details provided in requirement specifications in terms of transition, retry, timeout and various other microservice components. This process can however be automated. We choose a particular flow and specify tasks and workflows for them including all the minute details that are normally possible over network communicating of distributed systems. The equivalent Conductor specification of the microservice represents the complete workflow of the input and captures all the details regarding the various services. Next, we convert the Conductor specification into Petri nets (more specifically a timed Petri net) using the Conductor2PN framework [5].

We now do an analysis based on traversal on this converted Petri net. Traversal analysis is nothing but visiting each node like a graph based on various conditions while we unfold these Petri nets. In general, there are two kinds of analysis i.e. feasible execution path detection which is used to mainly detect liveness property and design for failure which is used to detect deadlock [4, 5]. Feasibility execution path refers to a traversal where in we start at a particular node and are able to end at some node. On the other hand, design for failure is a state where in we start at a node but are unable to come out due to various factors which is referred to as deadlock. Based on categorisation of systems explained above, either CTL specifications framed earlier or properties framed for SMT solver are verified on the various paths of converted Petri nets by feeding to model checking tool(s) like Graphgen [6] or Z3 solver. A counter example is provided if a property is not satisfied. For example, if we wish to check whether a microservice has run into a deadlock,

```
{
  "name": "Cancel_ticket",
  "description": "Cancels ticket and refunds amount",
  "version": 2,
  "schemaVersion": 2,
  "tasks": [
    {
      "name": "cancel_service",
      "taskReferenceName": "cancel_service",
      "inputParameters": {
        "contentId": "${workflow.input.orderId}"
      },
      "type": "SIMPLE"
    },
    {
      "name": "calculate_refund",
      "taskReferenceName": "calculate_refund",
      "inputParameters": {
        "contentId": "${cancel_service.input.calculate_refund}"
      },
      "type": "SIMPLE"
    },    {
      "name": "cancel_service",
      "taskReferenceName": "cancel",
      "inputParameters": {
        "contentId": "${calculate_refund.input.cancel}"
      },
      "type": "SIMPLE"
    },    {
      "name": "order_service",
      "taskReferenceName": "order_service",
      "inputParameters": {
        "contentId": "${cancel_service.input.order_service}"
      },
      "type": "SIMPLE"
    },
  ]
}
```


Figure 2: Workflow Definition for part of Train Ticket service using Conductor

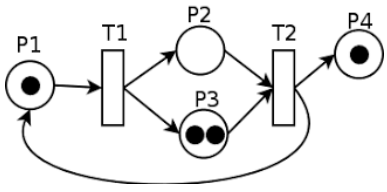


Figure 3: Petri Net Example [14]

we verify a CTL property where in we check whether end state is reached with all the possible traversals. If we get one such traversal path where in end node/place is not reachable, this path would be provided as a counter example. Counter examples are further analysed and are compared with the faults on benchmark system for proper mapping on the properties derived.

We have categorised systems into two categories i.e. Simple and Complex, which is done based on number of states in converted Petri net. Based on our experimental study on Petri nets, simple systems are those having 1000 states/places or less and the rest constitute complex systems. Our proposed verification approach for complex systems can be applied for simple systems as well but not vice versa.

Based on our analysis on the nature and characteristics of microservices flow explained in Section 2.1 for simple systems, we have framed CTL specifications which would later be used during traversal analysis to find out faults. To start with, we have formulated six properties on microservices specifications that can

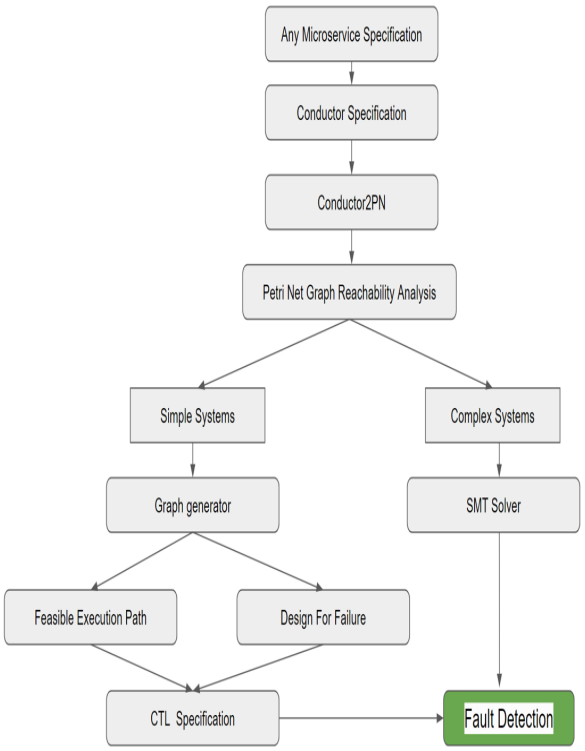


Figure 4: Proposed Methodology

be written as CTL formulas as explained in Section 2.5. "S" at the beginning of each property as prefix corresponds to simple systems while "C" corresponds to complex systems. Following are the properties identified:

- $\mathbf{SP}_1$ - Communication behavior across microservices can be verified through correctness of system. For example, receipt is one of the microservices(whose functionality is evident by virtue of its name) which follows payment microservice in a system where payment is mandatory to be successful: $AF(payment > 0 \&\& receipt > 0)$.
- $\mathbf{SP}_2$ - Failure of a microservice can be identified using the property: $EF(failurestate > 0)$. One of the states is identified as failure state.
- $\mathbf{SP}_3$ - To verify decentralized property of microservies, we need to verify: AF(end is reached) for all microservices within a system.
- $\mathbf{SP}_4$ - To verify feasible execution path i.e to verify whether the system has reached it's end state, we need to verify the following: AF(end is reached).
- $\mathbf{SP}_5$ - To verify whether the system involving microservices has reached deadlock state, we verify the following CTL formula: EF(end is not reached).
- $\mathbf{SP}_6$ - Bounded response time can be verified using the number of tokens i.e.: AG(some state) $< desiredtoken$.

On the other hand, complex systems are verified using SMT solver and following are the properties identified based on tokens in respective states/places which when passed through solver would produce result as satisfied or not satisfied.

- $\mathbf{CP}_1$ - Communication behavior across microservices for example stated in property SP1 can be verified using: $payment == 0 \&\& receipt > 0$.
- $\mathbf{CP}_2$ - Failure of a microservice can be identified using: failure state > 0.
- $\mathbf{CP}_3$ - To verify decentralized property, for all microservices, we need to verify: $end > 0$.
- $\mathbf{CP}_4$ - To verify feasible execution path, we need to verify the following: $end > 0$.
- $\mathbf{CP}_5$ - To verify whether the system involving microservices has reached deadlock state, we verify the using: $end == 0$.
- $\mathbf{CP}_6$ - Bounded response time can be verified using: there is some state such that $state < desiredtoken$.

5 RESULTS AND ANALYSIS

A microservices application called TrainTicket [11] is used as a benchmark to illustrate our proposed methodology. The application was released with a known set of faults and meant to be used as a benchmark for other methodologies related to testing and verification of microservices. Using our methodology, we were able to detect seven faults out of twenty two faults on train ticket system based on the properties explained in methodology. Other fifteen faults which could not be detected were either because of coding error or configuration error which cannot be captured at design phase. Our work mainly concentrates on finding faults at early stage and all the possible faults on benchmark system are detected which turns to be seven in number. Table 1 depicts the seven faults detected using our approach.

F1 is a fault which occurs due to incorrect sequence of asynchronous operations. For the TrainTicket system, Ticket-Cancel-Service, Inside-Payment-Service and Order-Service were used and tasks/workflows which also can be seen in Figures 1 and 2 were written for them to verify the properties. F2 is a fault where incorrect information is shown. For this, we used Train-service microservice and a random delay was introduced in workflow definition and the fault was replicated while verifying Property $\mathbf{SP}_3/\mathbf{CP}_3$. F4 is related to bounded response time and was verified with Train-service by introducing new state called ssl-offloading in tasks/workflow definition through property $\mathbf{SP}_6/\mathbf{CP}_6$. Similarly, F5 which is related to timeout was also verified using property $\mathbf{SP}_6/\mathbf{CP}_6$. F7 and F16 are related to failure tasks and respective microservices were used to define conductor tasks and workflows and the fault was detected using property $\mathbf{SP}_2/\mathbf{CP}_2$. F20 is a fault wherein no result was shown on executing train ticket system and this fault was identified using our deadlock property $\mathbf{SP}_5/\mathbf{CP}_5$ with the associated microservices of the system.

In this way, we were able to identify seven industrial faults on TrainTicket system using our proposed method.

6 CONCLUSION

It is essential to detect faults in microservice applications which have a higher industry adoption at an early stage to effectively utilise resources. We have proposed a methodology to identify faults on microservices at requirements stage. We have verified

Table 1: Mapping of TrainTicket faults with properties identified through our proposed method

Sl.No	Microservice Errors [10]	Mapping
1	F1 (Wrong order)	SP_1/CP_1
2	F2 (Some information is wrong)	SP_3/CP_3
3	F4 (Long response time)	SP_6/CP_6
4	F5 (Timeout)	SP_6/CP_6
5	F7 (Service failure)	SP_2/CP_2
6	F16 (Operation Failure)	SP_2/CP_2
7	F20 (No Response)	SP_5/CP_5

this on the train ticket benchmark system and our method shows satisfactory results. We plan to extend this work as a part of formal verification of microservices and try to mitigate as many errors as possible at an early requirement and design phase and thus paving way for greater resource utilisation across industry.

REFERENCES

[1] 2017. CONDUCTOR TO PN. Retrieved March 21, 2022 from https://github.com/SELab-unimi/conductor2pn

[2] Christel Baier and Joost-Pieter Katoen. 2008. *Principles of model checking.* MIT press.

[3] Pere Bonet, Catalina M Lladó, Ramon Puijaner, William J Knottenbelt, et al. 2007. PIPE v2. 5: A Petri net tool for performance modelling. In *Proc. 23rd Latin American Conference on Informatics (CLEI 2007).*

[4] Matteo Camilli, Carlo Bellettini, Lorenzo Capra, and Mattia Monga. 2016. Coverability analysis of time basic Petri nets with non-urgent behavior. In *2016 18th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC).* IEEE, 165–172.

[5] Matteo Camilli, Carlo Bellettini, Lorenzo Capra, and Mattia Monga. 2018. A Formal Framework for Specifying and Verifying Microservices Based Process Flows. In *Software Engineering and Formal Methods*, Antonio Cerone and Marco Roveri (Eds.). Springer International Publishing, Cham, 187–202.

[6] Matteo Camilli and Carlo Nicola Maria Bellettini. 2012. Graphgen: Java command line tool for state space analysis of Time-Basic (TB) Petri nets. (2012).

[7] Qingfeng Du, Tiandi Xie, and Yu He. 2018. Anomaly detection and diagnosis for container-based microservices with performance monitoring. In *International Conference on Algorithms and Architectures for Parallel Processing*. Springer, 560–572.

[8] Martin Fowler. 2014. Microservices. Retrieved March 21, 2022 from https://martinfowler.com/articles/microservices.html

[9] Lun Meng, Feng Ji, Yao Sun, and Tao Wang. 2021. Detecting anomalies in microservices with execution trace comparison. *Future Generation Computer Systems* 116 (2021), 291–301.

[10] Microservice System Benchmark 2018. Train Ticket Fault Replicate. Retrieved March 21, 2022 from https://github.com/FudanSELab/train-ticket-fault-replicate

[11] Microservice System Benchmark 2018. Trainticket. Retrieved March 21, 2022 from https://github.com/FudanSELab/train-ticket/

[12] Netflix, Inc. 2016. Conductor. Retrieved March 21, 2022 from https://netflix.github.io/conductor/

[13] Netflix Technology Blog 2016. Netflix Conductor: A microservices orchestrator. Retrieved March 21, 2022 from https://netflixtechblog.com/netflix-conductor-a-microservices-orchestrator-2e8d4771bf40

[14] Wolfgang Reisig. 2012. *Petri nets: an introduction.* Vol. 4. Springer Science & Business Media.

[15] Statista 2021. Organizations' adoption level of microservices worldwide in 2021. Retrieved March 21, 2022 from https://www.statista.com/statistics/1233937/microservices-adoption-level-organization/

[16] Tao Wang, Wenbo Zhang, Jiwei Xu, and Zeyu Gu. 2020. Workflow-aware automatic fault diagnosis for microservice-based applications with statistics. *IEEE Transactions on Network and Service Management* 17, 4 (2020), 2350–2363.

[17] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. 2019. Latent error prediction and fault localization for microservice applications by learning from system trace logs. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 683–694.